

Reflexion und kritische Analyse

Am Anfang hatten wir ehrlichen Respekt davor, eine komplette App mit Login, Datenbank und KI aufzubauen. Keiner von uns hatte vorher ein produktionsnahes Backend gebaut, und entsprechend unsicher war das Gefühl zu Beginn. Rückblickend war genau das der wertvollste Teil des Projekts. Wir mussten die App nicht nur zum Laufen bringen, sondern auch verstehen, warum sie läuft und an welchen Stellen sie angreifbar wird.

Wie ist die IT-Security?

In der Vorlesung haben wir die typischen Schwachstellen von KI-generiertem Code kennengelernt. Dazu gehören fehlende Autorisierung, SQL-Injection, hartkodierte Zugangsdaten und offene Debug-Endpoints. Wir haben versucht, diese Punkte von Anfang an bewusst mitzudenken, statt darauf zu vertrauen, dass die KI das schon richtig macht.

Was wir gut gelöst haben

- **Autorisierung über Row Level Security, nicht über das Frontend.** Die Datenbank ist unsere Sicherheitsgrenze. RLS ist auf allen fachlichen Tabellen aktiv und umfasst rund siebzig Policies. Jede Abfrage ist auf den eigenen Nutzer eingeschränkt. Filter im Frontend behandeln wir ausdrücklich nicht als Schutz. Wer die API direkt anspricht, sieht trotzdem nur seine eigenen Daten. Damit haben wir den häufigsten Vibe-Coding-Fehler vermieden, nämlich die fehlende Autorisierung.
- **Keine Secrets im Code.** Die Datei mit den geheimen Werten ist von der Versionsverwaltung ausgeschlossen. Im Repository liegt nur eine Beispieldatei ohne echte Inhalte. Der sensible Service-Role-Key wird lokal erzeugt und mit eingeschränkten Dateirechten gespeichert, sodass er nie eingecheckt wird. Der öffentliche Anon-Key darf bewusst öffentlich sein, weil die eigentliche Absicherung über RLS läuft.
- **Schutz vor SQL-Injection.** Wir arbeiten ausschließlich über den Supabase-Client und parametrisierte Funktionen. SQL-Befehle bauen wir nie als Text zusammen, wodurch die klassische Angriffsfläche wegfällt.
- **Solide Grundlagen bei der Anmeldung.** Die Bestätigung per E-Mail ist Pflicht und für Passwörter gilt eine Mindestlänge. Der Schutz von Sitzungen und Routen läuft serverseitig über den Proxy und nicht nur über ein Flag im Frontend.

- **Private Dateien.** Uploads liegen in einem privaten Storage-Bucket mit nutzerbezogenen Policies und einem festen Ablauf für das Aufräumen. Für diesen Bereich gibt es eigene Datenbanktests.
- **Automatisierte Kontrolle.** Unsere CI prüft bei jeder Änderung den Stil, die Tests und einen frischen Build.

Wo wir ehrlich noch Lücken sehen

- **Kein echter Penetrationstest.** Unser Sicherheitsblick bestand vor allem aus gegenseitigem Code-Lesen und gezielten Nachfragen an die KI. Das ersetzt keinen strukturierten Test.
- **Threat Modeling nur im Kopf.** Mögliche Angriffswege haben wir mitgedacht, aber nicht sauber dokumentiert. Für ein echtes Produkt würden wir das vorher aufschreiben.
- **Dünnere Schutz vor Missbrauch und Kosten.** Beim KI-Chat fehlen harte Rate-Limits. Die Geschichte aus der Vorlesung mit der versehentlichen Rechnung in Millionenhöhe zeigt, wie schnell ein offener KI-Endpoint teuer werden kann.
- **Abhängigkeit von Supabase.** Vieles steht und fällt mit einem Anbieter. Für ein Studienprojekt ist das in Ordnung, für ein echtes Startup wäre diese Abhängigkeit ein eigener Risikopunkt.

Unterm Strich halten wir die Sicherheitsbasis für solide, gerade weil die Autorisierung von Anfang an in der Datenbank liegt. Perfekt ist sie nicht, aber wir können zu jeder Entscheidung sagen, warum wir sie so getroffen haben.

Was haben wir gelernt?

- **KI ist ein schneller Junior, kein Senior.** Sie liefert in Minuten etwas Lauffähiges, widerspricht aber nie und warnt nicht vor schlechten Ideen. Vorangekommen sind wir erst, als wir selbst geplant und die KI gezielt zur Umsetzung genutzt haben.
- **Review ist der eigentliche Skill.** Code erzeugen ist leicht geworden. Ihn zu lesen, zu verstehen und Schwachstellen zu finden ist die eigentliche Arbeit und genau das, was am Ende wirklich hängen bleibt.
- **Erst Architektur, dann Prompt.** Unsere besten Ergebnisse entstanden, wenn wir vorher über Frontend, Backend und Datenmodell nachgedacht haben. Ohne Plan beginnt der Code zu wuchern.
- **Sicherheit gehört von Anfang an dazu.** Die Regeln früh in die Datenbank zu legen war viel einfacher, als Autorisierung später nachzurüsten.

Was würden wir anders machen?

- **Scope früher eingrenzen.** An einigen Stellen haben wir mehr angefangen, als wir am Ende sauber fertig bekamen. Weniger Funktionen, dafür vollständig und rund, wäre die bessere Wahl gewesen.
- **Tests und CI ab Tag eins.** Wir haben beides erst später ernst genommen. Früher wäre vieles stabiler und schneller gelaufen.
- **Entscheidungen direkt dokumentieren.** Einiges mussten wir im Nachhinein rekonstruieren, weil wir nicht festgehalten hatten, warum wir einen bestimmten Weg gewählt haben.
- **Fester Security-Durchgang.** Statt Sicherheit nebenbei mitlaufen zu lassen, würden wir einen festen Termin einplanen, mit einer Begrenzung der Zugriffe und einem kleinen, bewussten Test zum Abschluss.

Das größte Learning ist eigentlich simpel. Vibe Coding macht uns schnell, aber nicht automatisch gut. Gut werden wir erst dadurch, dass wir verstehen, was dort entstanden ist, und den Mut haben, es auch wieder infrage zu stellen.